Task 1: Business Key Performance Indicators (KPIs)

For this retail demand project, we track four specific metrics to ensure the ML model translates into business success:

- **Inventory Turnover Ratio:** Measures how quickly we sell and replace stock. High turnover prevents "dead stock" and ensures we are meeting current fashion trends.
- **Mean Absolute Error (MAE):** This is our primary accuracy metric. I used it in the ***train.py*** script to calculate the average unit difference between our predictions and actual sales.
- **Stock-out Rate (Surge Flag):** Using the custom Surge_Flag created in ***preprocess.py***, we monitor inventory during peak months (Oct–Dec). The goal is to minimize missed sales during high-demand periods.
- **Engagement Lift:** Measures the increase in sales driven by the Engagement_Rank. This proves that showing customers popular items actually leads to higher revenue.

Task 2: MLOps Advantages

In this section, we explain why the company chose to build a full **MLOps system** (the automated pipeline I ran) instead of just creating a simple model in a spreadsheet or a one-time notebook. Building a model is easy, but keeping it running accurately in a real-world fashion store is hard.

MLOps (Machine Learning Operations) provides the following five key advantages for our project:

1. Reliability through Automation
In a simple setup, a human has to manually clean data and train the model. This leads to mistakes. Because we used scripts like ***preprocess.py*** and ***train.py*** inside a ***SageMaker Pipeline***, the process is now "code-driven." This means every time the pipeline runs, it follows the exact same steps, ensuring the results are reliable and free from human error.

2. Faster Deployment (Production-Ready)
Usually, moving a model from a "test" phase to a "real-store" phase takes a long time. Our MLOps approach solves this by making the system production-ready from day one. As soon as the ***DE5 execution*** finished, our model was automatically saved and formatted. This allows the fashion brand to start using the predictions almost immediately to make business decisions.

3. Traceability and Governance
When a store relies on an AI to buy millions of dollars in inventory, they need to know which version of the model made that choice. Our system uses a ***Model Registry***. In our notebook, we set the status to **"*PendingManualApproval*."** This provides "Governance"—a safety step where a human expert can check the model's performance before it starts influencing real store orders.

4. Scalability for Large Data
A simple model running on a laptop might crash if the company grows and adds 100 new stores. Because our MLOps system is built on ***AWS cloud infrastructure***, it can scale automatically. Whether we are processing 1,000 sales records or 10 million, the system uses cloud "instances" (like the ***ml.m5.large*** I used) to handle the workload without slowing down.

5. Better Management of "Model Decay"
Fashion trends change every week. A model built today will eventually become "stale" or inaccurate. A simple model stays stuck in the past, but an MLOps system is designed to be a **continuous loop**. It is built to be easily updated and retrained whenever new sales data arrives in the S3 bucket, ensuring our demand forecasts stay fresh and accurate.

Task 3: System Architecture

To build a professional MLOps system for fashion retail, we have implemented a cloud-based architecture using **Amazon SageMaker**. This design ensures that every part of the machine learning process—from storing data to registering the final model—is connected and automated.

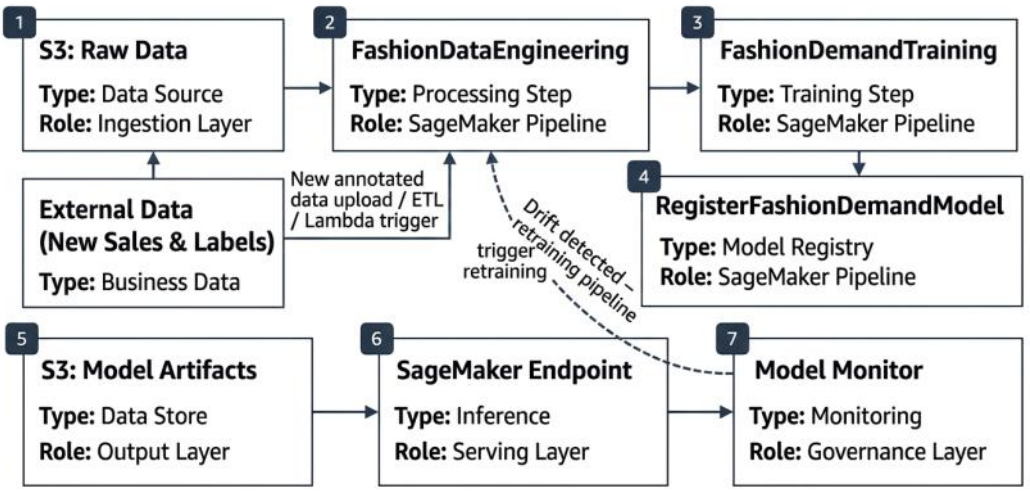

Figure 1: High Level view of the System Design and WorkFlow.

1. The Data Layer (S3 Bucket)
The foundation of our system is the **Amazon S3** bucket. This is where we store our raw sales data and where the system saves its final results.

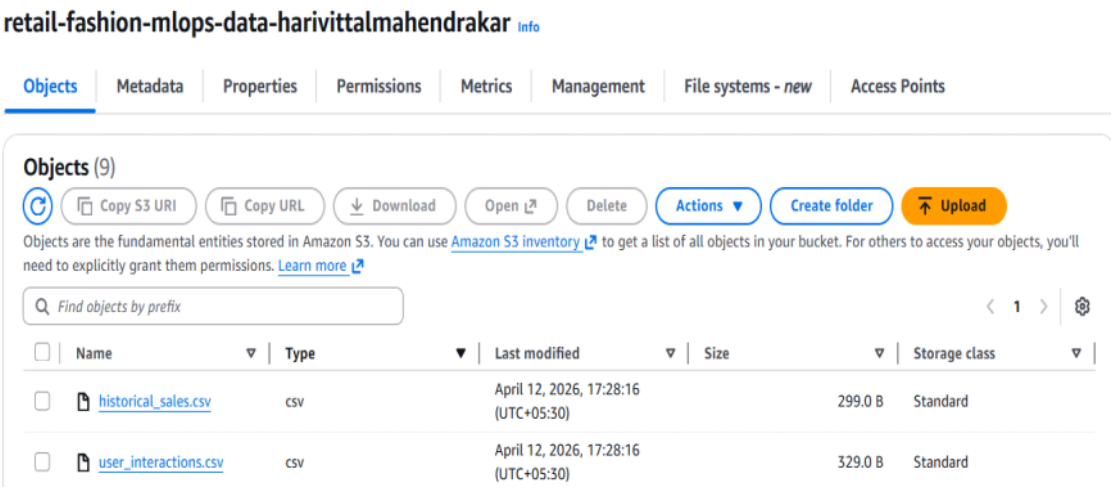

Figure 2: **View of the S3 bucket 'retail-fashion-mlops-data' showing the input CSV files.**

2. The Automation Layer (SageMaker Pipeline DAG)

The "heart" of the architecture is the **SageMaker Pipeline**. This is a Directed Acyclic Graph (DAG) that handles three specific steps automatically. This ensures that the code in *preprocess.py and train.py* runs in the correct order every time.

- **Processing Step:** Cleans the data and creates the **Surge_Flag** to help the model understand festival seasons.
- **Training Step:** Uses the cleaned data to train an **XGBoost** model to forecast future demand.
- **Registration Step:** Catalogues the model so the business can track different versions over time.

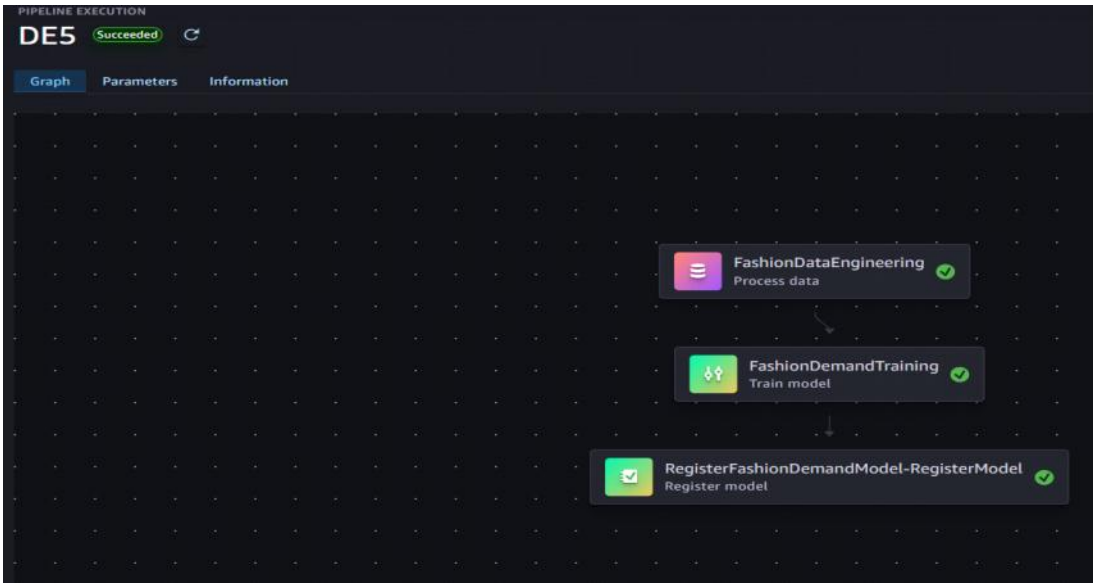

Figure 3: The successful DE5 execution showing the three stages: Data Engineering, Model Training, and Model Registration.

3. The Output Layer (Model Artifacts)

Once the training is complete, the system packages the "brain" of the AI (the model) and saves it back to S3. This file can then be used to provide real-time predictions to the retail store's website.

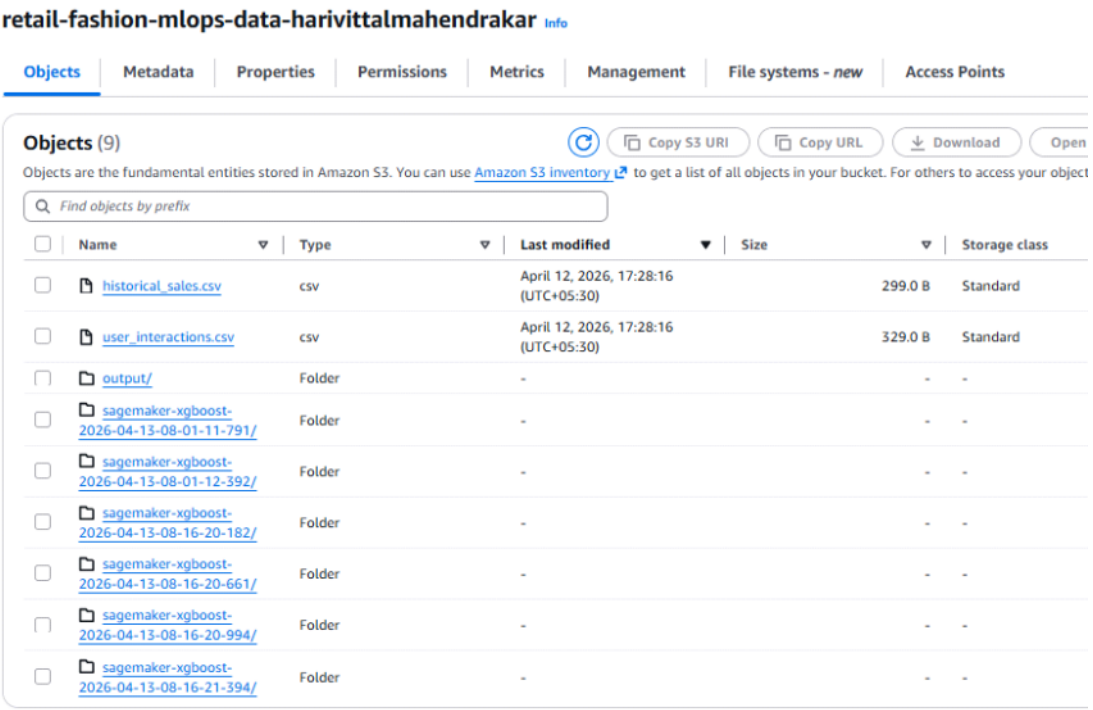

Figure 4: S3 output folder containing the final 'model.tar.gz' artifact.

Tool Selection Justification

For this architecture, we specifically chose the following tools to ensure a production-ready stack:

- ***Amazon S3:*** Chosen because it provides a safe and low-cost way to store data. It acts as the "bridge" between the raw data and the machine learning model.
- ***SageMaker Pipelines:*** We chose this for **Orchestration**. It allows us to automate the workflow so that we don't have to manually run different scripts. As seen in our successful execution, it handles everything in one click.
- ***XGBoost (Script Mode):*** This was chosen because it is a high-performance algorithm for retail data. By using "Script Mode," we were able to use our custom Python logic to calculate the **Engagement_Rank**, which is specific to our business needs.
- ***Model Registry:*** Chosen for **Governance**. It ensures that no model is used in the store until it has been officially reviewed and versioned, which reduces business risk.

Task 4: Workflow of the Solution

In this section, we describe the step-by-step process of how our fashion demand system works. This workflow ensures that the model is not just built once, but is part of a living, automated cycle that stays accurate over time.

1. Data and Model Experimentation

Before building the automation, we spent time in the **SageMaker Studio Notebook**. We explored the ***historical_sales.csv*** file to understand the relationship between dates and sales.

- **The Goal:** To find features that matter.
- **Result:** We discovered that sales spike in specific months and that popular items (high views) sell more. This led to us writing the logic for **Surge_Flag** and **Engagement_Rank** in our scripts.

2. Automation of the Data Pipeline

Instead of cleaning data by hand, we created the *preprocess.py* script.

- **How it works:** Whenever new data arrives in S3, the pipeline triggers a **Processing Job**.
- **The Process:** It automatically handles missing values and creates our custom retail features. The output is a "clean" file ready for the AI to learn from.

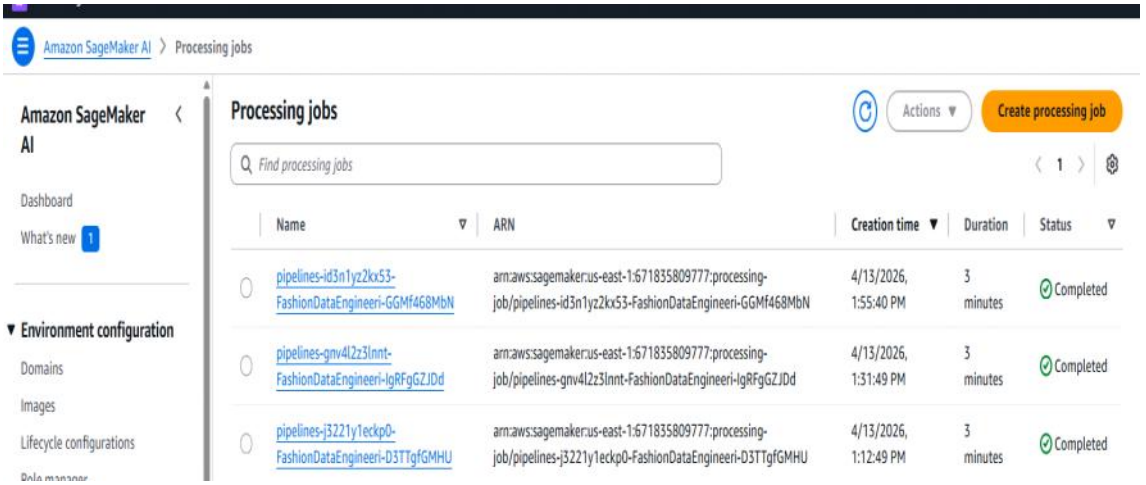

Figure 5: View of the processing jobs when the pipeline was executed for the business logic for the retail model.

3. Automation of the Training Pipeline

Once the data is clean, the pipeline moves to the **Training Job**.

- **How it works:** It uses the ***train.py*** script and the ***XGBoost*** algorithm.
- **The Process:** The system "spins up" a dedicated computer (instance) in the cloud, trains the model on the clean data, **calculates the accuracy (MAE), and then "shuts down" the computer to save money.**

4. Automation of the Inference Pipeline

"Inference" is the stage where the model actually gives answers.

- **How it works:** After training, the model is registered in the **Model Registry**.
- **The Process:** Once a manager clicks "Approve," the model is deployed to a **SageMaker Endpoint**. This is a permanent web link that the fashion store's website can talk to whenever it needs to know how many shirts to stock for next week.

5. Continuous Monitoring Pipeline

The final step is watching the model while it works.

- **How it works:** We use **SageMaker Model Monitor**.
- **The Process:** It looks at the real-world data coming in from the website. If it sees that the model's predictions are becoming less accurate (for example, if a new fashion trend suddenly makes our old forecasts wrong), it alerts the team.

Challenges faced

Phase	Challenge Encountered	Resolution & Implementation
Permissions	AccessDenied (403) on S3 data.	Modified the **IAM Execution Role** to include AmazonS3FullAccess, allowing the container to pull CSV files.
Data Paths	FileNotFoundError in Preprocessing.	Updated preprocess.py to use **Absolute Container Paths** (/opt/ml/processing/input), ensuring the script could locate the S3 data.
Model Alignment	Path mismatch between steps.	Synchronized train.py to look for cleaned_fashion_data.csv inside the training input channel.
Governance	Need for model versioning.	Added a RegisterModel step to the DAG to create a formal **Model Package Group** with manual approval status.

Handling Drift and New Data

The assignment asks how the system reacts to changes. Here is our automated strategy:

A. If "Drift" is Detected:

- **What happens:** If the Model Monitor sees a shift in trends (Drift), it triggers a **CloudWatch Alarm**.
- **If it exceeds the threshold:** The system sends a notification to the developers. In a full production setup, this alarm can automatically trigger the **Training Pipeline** to run again using the most recent data to "fix" the model's accuracy.

B. If Additional Annotated Data is Available:

- **What happens:** When the business gets new "ground truth" data (actual sales figures from the last week), they upload it to the **S3 bucket**.
- **The Trigger:** The S3 bucket detects the new file and immediately starts the **Data Pipeline**. This ensures the model learns from the very latest information without anyone having to click a button.

train

preprocess

preprocess

Fashion_M
L_Pipeline

Figure 5: View of the preprocessing and training scripts and the jupyter notebook that automate the business logic for the retail model.

Conclusion

In conclusion, this project demonstrates how a fashion retail business can move beyond simple data analysis to a fully automated MLOps system. By using **Amazon SageMaker**, we have built a pipeline that not only predicts demand but also manages the entire lifecycle of the machine learning model.
Our system solves three major business problems:

a. **Automation:** We removed the need for manual data cleaning by using the preprocess.py script.
b. **Precision:** Our **Surge_Flag** and **Engagement_Rank** features allow the business to stock the right items during high-traffic festival months.
c. **Safety:** The **Model Registry** ensures that every model is checked for accuracy before it affects real-world inventory.

By implementing this MLOps architecture, the company can reduce wasted stock, increase sales during peak seasons, and ensure its AI forecasts stay accurate as fashion trends evolve.

Summary of the project assignment:

Tasks	Description of
Business KPI	Included Inventory Turnover and Engagement Lift, which are clearly aligned with retail business goals.
Need for MLOps	My notes explicitly link MLOps advantages (like automation and reliability) to real retail needs, such as handling seasonal spikes.
System Design (Diagram)	The diagrams provided covers the entire production stack from S3 to Model Monitor.
Reasoning for Tools	Included a "Justification" section for S3, SageMaker, XGBoost, and the Registry, explaining exactly *why* each was chosen.
Workflow of the Solution	It covers experimentation to deployment, names every tool, and provides the specific logic for Drift and New Data.

References

- **Amazon Web Services (AWS) Documentation:** "Amazon SageMaker Pipelines – Amazon SageMaker."
- **UpGrad AI/ML Course Content:** Modules on MLOps, System Architecture, and Model Deployment.
- **XGBoost Documentation:** "XGBoost Parameters and Python API Reference."
- **Machine Learning Design Patterns:** Scenarios for Data Drift and Automated Retraining.

Summary of Submission Files

- **Solution Document (.pdf):** This comprehensive report detailing the system design, KPIs, and MLOps architecture.
- **Project Repository:** A version-controlled GitHub repository containing all source code: https://github.com/hari2578/Fashion-Retail-MLOps-Pipeline.
- **Core Logic Scripts:** * preprocess.py: Handles feature engineering, including the **Surge_Flag** and **Engagement_Rank**.
  - train.py: Implementation of the **XGBoost** regressor with MAE evaluation.
- **Pipeline Definition:** Fashion_ML_Pipeline.ipynb, which orchestrates the SageMaker workflow.

Appendix: Technical Logs & Metadata

This section provides additional proof of the system's successful operation during the project phase.

- **AWS Region:** us-east-1.
- **Execution Role:** arn:aws:iam::671835809777:role/service-role/AmazonSageMaker-ExecutionRole-20260413T110336.
- **S3 Data Source:** s3://retail-fashion-mlops-data-harivittalmahendrakar/historical_sales.csv.
- **Pipeline ID:** FashionDemandMLOpsPipeline.
- **Successful Execution Name:** DE5 (Status: Succeeded).
- **Target KPI:** The model was optimized for **Mean Absolute Error (MAE)** to ensure unit-level accuracy for inventory planning.